Actividad 2.6 - Dashboard de métricas

Documentación técnica del entregable | 10 de octubre de 2026

Estructura hexagonal

```
backend/src/
  domain/ports/AnalyticsRepositoryPort.js          Puerto de salida
  application/use-cases/AnalyticsService.js        Puerto de entrada / caso de uso
  infrastructure/repositories/AnalyticsRepositoryAdapter.js  PostgreSQL
  infrastructure/http/routes/analyticsRoutes.js    Controlador y ruta Express
  bootstrap/createServices.js                      Inyección de dependencias
frontend/src/
  application/analytics.js                         Servicio de aplicación
  infrastructure/http/AnalyticsGateway.js          Consumo de API
  presentation/DashboardAdmin.jsx                  Vista, filtro temporal y gráficos Recharts
  presentation/DashboardAdmin.css                  Diseño adaptable
```

El subsistema consulta `shop_orders` y `shop_order_items`, sin cambiar entidades operativas. Los pedidos de proveedores en `orders` no representan ventas de tienda y se excluyen. Solo se agregan índices de consulta a la migración existente.

API

`GET /api/analytics/dashboard` requiere `Authorization: Bearer <token>` y rol `super_admin`, el administrador existente. La sesión se verifica con el mecanismo actual.

Parámetro	Valores	Predeterminado
period	daily, weekly, monthly, custom	weekly
startDate	YYYY-MM-DD	obligatorio para custom
endDate	YYYY-MM-DD	obligatorio para custom
top	5 o 10	5

Si se proporciona una fecha se requieren ambas; estas prevalecen sobre el periodo. daily corresponde a hoy; weekly a hoy y los seis días anteriores; monthly al primer día del mes hasta hoy. Fechas UTC, ambas inclusivas, máximo 366 días. Se filtra por fecha de creación del pedido, no por fecha del pago. El filtro SQL utiliza inicio inclusivo y siguiente medianoche exclusiva.

Ejemplos: `/api/analytics/dashboard?period=monthly&top=10` y `/api/analytics/dashboard?period=custom&startDate=2026-10-01&endDate=2026-10-10`.

Respuesta 200 (ejemplo ilustrativo):


```
{
  "range": {"startDate":"2026-10-10","endDate":"2026-10-10","timezone":"UTC"},
  "currency":"MXN",
  "summary":{"totalRevenue":250,"paidOrders":1,"customers":1,"averageOrder":250,"averageUser
":250},
  "trend":[{"date":"2026-10-10","revenue":250}],
  "orderStates":[{"state":"Cancelado","count":0,"percentage":0},{"state":"Enviado","count":0
,"percentage":0},{"state":"Pagado","count":1,"percentage":100},{"state":"Pendiente","count":
0,"percentage":0}],
  "topProducts":[{"productId":1,"name":"Producto ejemplo","units":2,"revenue":200}]
}
```

Errores: 400 por filtros inválidos; 401 por token ausente, inválido o expirado; 403 por rol sin permisos; 500 por fallo interno. Cuerpo: `{"message":"Descripción"}`.

Definiciones de las métricas

- Ventas: pedidos con `payment_status=pagado` y estado distinto de `cancelado`; se excluyen pendientes y reembolsados.

- Ingresos totales: suma de `total_cents`, incluye envío. Ticket por pedido: ingresos / pedidos pagados. Gasto por usuario: ingresos / compradores distintos en el rango (no promedio por pedido de cada usuario).

- Ranking: suma de cantidades e importe histórico `quantity * unit_price_cents`, excluye envío. Agrupa por ID y nombre histórico; un producto renombrado puede aparecer en filas distintas, y los eliminados conservan su nombre histórico. Orden por unidades, ingresos y nombre.

- Estados, para todos los pedidos del rango: Cancelado tiene prioridad; Enviado agrupa enviado y entregado; Pagado agrupa los restantes pagados; Pendiente agrupa el resto. Cada pedido aparece una vez. Los porcentajes se redondean a dos decimales.

- Días sin ventas, métricas vacías y categorías sin pedidos se devuelven como cero.

Las sumas, conteos, promedios, clasificación, ranking, porcentajes y serie diaria se calculan en PostgreSQL en una sola consulta parametrizada con una instantánea consistente. No se cargan pedidos individuales en Node.js. Los importes se almacenan en centavos y la API los devuelve en pesos MXN.

Uso local

Ejecutar `npm run db:setup` en backend para crear índices; iniciar con `npm run dev` desde frontend (inicia API y Vite). Entrar con una cuenta super_admin y elegir **Dashboard de métricas** en el portal. El filtro reconsulta automáticamente; el ranking permite elegir Top 5/10, buscar y ordenar columnas. La vista contempla carga, error, reintento y datos vacíos. No se requiere despliegue para esta actividad.

Validación realizada

47 pruebas de backend, incluidas agregaciones en PostgreSQL y permisos HTTP; 8 pruebas de frontend aprobadas. Lint y compilación de producción correctos. Los índices se aplicaron a la base de datos local.

Código y documentación en GitHub

Repositorio: github.com/TitanLopezz/Markettaller. El código completo está en backend/src y frontend/src; las pruebas en backend/test. Este documento acompaña la implementación de la Actividad 2.6.